HOSPITAL RECEIVER

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sched.h>
#include <time.h>
#include <sys/neutrino.h>

#define CRITICAL 1
#define ROUTINE 2
#define INTERRUPTED 3

#define HIGH_PRIORITY 30
#define LOW_PRIORITY 10

#define HR 1
#define SPO2 2
#define TEMPERATURE 3
#define GPS 4
#define EMERGENCY 5

typedef struct {
    int ambulance_id;
    int parameter;
    int priority;
    int severity;
    double value;
    double latitude;
```

```
    double longitude;
} PatientMessage;

int critical_chid;
int routine_chid;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
unsigned long critical_count = 0;

const char *parameter_name(int parameter) {
    switch (parameter) {
        case HR: return "HEARTBEAT";
        case SPO2: return "SpO2";
        case TEMPERATURE: return "TEMPERATURE";
        case GPS: return "GPS";
        case EMERGENCY: return "EMERGENCY SEVERITY";
        default: return "UNKNOWN";
    }
}

void display_message(PatientMessage *msg) {
    printf("\nAmbulance ID: %d\n", msg->ambulance_id);
    printf("Parameter: %s\n", parameter_name(msg->parameter));

    switch (msg->parameter) {
        case HR:
            printf("Heartbeat: %.0f BPM\n", msg->value);
            break;
        case SPO2:
            printf("SpO2: %.0f%%\n", msg->value);
            break;
```

```
        case TEMPERATURE:
            printf("Temperature: %.1f C\n", msg->value);
            break;
        case GPS:
            printf("GPS: %.6f, %.6f\n",
                msg->latitude, msg->longitude);
            break;
        case EMERGENCY:
            printf("Emergency severity: %d/5\n",
                msg->severity);
            break;
    }
}

void set_priority(int priority) {
    struct sched_param param;
    memset(&param, 0, sizeof(param));
    param.sched_priority = priority;

    if (pthread_setschedparam(pthread_self(),
                    SCHED_RR, &param) != 0) {
        perror("pthread_setschedparam");
        printf("Check QNX scheduling permissions.\n");
        exit(EXIT_FAILURE);
    }

    printf("SCHED_RR priority %d activated\n", priority);
}

void *critical_receiver(void *arg) {
    PatientMessage msg;
```

```
int rcvid;
int reply;

set_priority(HIGH_PRIORITY);

while (1) {
   memset(&msg, 0, sizeof(msg));
   rcvid = MsgReceive(critical_chid, &msg,
                sizeof(msg), NULL);

   if (rcvid == -1) {
      perror("Critical MsgReceive");
      continue;
   }

   if (rcvid == 0)
      continue;

   if (msg.priority != CRITICAL) {
      reply = -1;
      MsgReply(rcvid, 0, &reply, sizeof(reply));
      continue;
   }

   pthread_mutex_lock(&lock);
   critical_count++;
   pthread_mutex_unlock(&lock);

   printf("\n================================\n");
   printf("!!! CRITICAL ALERT RECEIVED !!!\n");
   display_message(&msg);
```

```
        printf("Processed by HIGH PRIORITY thread\n");
        printf("================================\n");
        fflush(stdout);

        reply = CRITICAL;
        MsgReply(rcvid, 0, &reply, sizeof(reply));
    }

    return NULL;
}

void *routine_receiver(void *arg) {
    PatientMessage msg;
    int rcvid;
    int reply;
    int i;
    int interrupted;
    unsigned long start_count;
    struct timespec delay = {0, 100000000};

    set_priority(LOW_PRIORITY);

    while (1) {
        memset(&msg, 0, sizeof(msg));
        rcvid = MsgReceive(routine_chid, &msg,
                    sizeof(msg), NULL);

        if (rcvid == -1) {
            perror("Routine MsgReceive");
            continue;
        }
```

```
if (rcvid == 0)
   continue;

if (msg.priority != ROUTINE) {
   reply = -1;
   MsgReply(rcvid, 0, &reply, sizeof(reply));
   continue;
}

pthread_mutex_lock(&lock);
start_count = critical_count;
pthread_mutex_unlock(&lock);

printf("\n--------------------------------\n");
printf("ROUTINE PROCESSING STARTED\n");
display_message(&msg);
fflush(stdout);

interrupted = 0;

for (i = 0; i < 50; i++) {
   nanosleep(&delay, NULL);

   pthread_mutex_lock(&lock);
   if (critical_count != start_count)
      interrupted = 1;
   pthread_mutex_unlock(&lock);

   if (interrupted)
      break;
```

```
        }

        if (interrupted) {
            printf("\nROUTINE INTERRUPTED!\n");
            printf("Critical alert took priority.\n");
            reply = INTERRUPTED;
        } else {
            printf("\nROUTINE PROCESSING COMPLETED\n");
            reply = ROUTINE;
        }

        fflush(stdout);
        MsgReply(rcvid, 0, &reply, sizeof(reply));
    }

    return NULL;
}

int main(void) {
    pthread_t high_thread;
    pthread_t low_thread;

    setvbuf(stdout, NULL, _IONBF, 0);

    critical_chid = ChannelCreate(_NTO_CHF_FIXED_PRIORITY);
    routine_chid = ChannelCreate(_NTO_CHF_FIXED_PRIORITY);

    if (critical_chid == -1 || routine_chid == -1) {
        perror("ChannelCreate");
        return EXIT_FAILURE;
    }
```

```
    printf("\nSMART AMBULANCE HOSPITAL RECEIVER\n");
    printf("Hospital PID: %d\n", getpid());
    printf("Critical Channel ID: %d\n", critical_chid);
    printf("Routine Channel ID: %d\n", routine_chid);

    if (pthread_create(&high_thread, NULL,
                critical_receiver, NULL) != 0) {
        printf("Failed to create critical thread\n");
        return EXIT_FAILURE;
    }

    if (pthread_create(&low_thread, NULL,
                routine_receiver, NULL) != 0) {
        printf("Failed to create routine thread\n");
        return EXIT_FAILURE;
    }

    pthread_join(high_thread, NULL);
    pthread_join(low_thread, NULL);

    return EXIT_SUCCESS;
}
```